

Kazalo vsebine

1. Uvod za developerje	3
1.1 Predznanje	3
1.2 Ključni koncepti	3
2. Setup lokalnega okolja	4
2.1 Predpogoji	4
2.2 Namestitve	4
2.3 VS Code priporočene razširitve	5
2.4 Debug setup	5
3. Arhitektura in struktura projekta	6
3.1 Direktorijška struktura	6
3.2 Ključni moduli	6
4. Kodni standardi	8
4.1 TypeScript pravila	8
4.2 React komponente	8
4.3 API Routes	9
5. Testiranje	11
5.1 Test hierarhija	11
5.2 Unit testi (Vitest)	11
5.3 E2E testi (Playwright)	12
5.4 Test pokritost cilji	12
6. Contribution workflow	13

6.1 Branching strategija	13
6.2 Commit convention	13
6.3 Pull Request postopek	13
6.4 PR checklist	14
7. Debug in troubleshooting	15
7.1 Pogoste težave in rešitve	15
7.2 Debug orodja	15
7.3 Performance profiling	16
8. Deployment	17
8.1 Environment-i	17
8.2 Deploy postopek	17
8.3 Environment variables	17
8.4 Pred-deploy checklist	18
9. Viri in nadaljnje učenje	19
9.1 Notranja dokumentacija	19
9.2 Zunanji viri	19
9.3 Kontakti	19

1. Uvod za developerje

Ta vodič je namenjen novim developerjem, ki se pridružujejo RestaurantOS ekipi. Vsebuje vse potrebno za hitri začetek: namestitev okolja, razumevanje arhitekture, kodne standarde, testiranje, contribution workflow in deployment postopke.

Po prebiranju tega vodiča bo nov developer sposoben: (1) nastaviti lokalno razvojno okolje v 30 minutah, (2) razumeti arhitekturo in module, (3) pisati kodo v skladu s standardi, (4) poganjati testi in debug, (5) prispevati PR-je v skladu z workflow.

30	92	229	659
min do prvega build-a	Prisma modelov	API rutin	React komponent

1.1 Predznanje

Pred začetkom dela na RestaurantOS priporočamo naslednje predznanje:

- **TypeScript** - osnovno znanje (interface, type, generic, async/await)
- **React 19** - hooks (useState, useEffect, useMemo), Server Components, Client Components
- **Next.js 16** - App Router, API Routes, file-based routing, layouts
- **PostgreSQL** - osnovni SQL, joins, indexes, transakcije
- **Prisma ORM** - schema definition, queries, migrations
- **Tailwind CSS 4** - utility classes, responsive design
- **Git** - branching, merging, rebasing, conflict resolution

1.2 Ključni koncepti

- **Multi-tenant arhitektura** - vsaka lokacija je tenant z locationId izolacijo (glej ADR-004)
- **FURS davčno potrjevanje** - ZOI, EOR, .p12 certifikat, offline queue (glej ADR-006)
- **PWA offline** - Service Worker, IndexedDB, Background Sync (glej ADR-007)
- **Optimistic locking** - updatedAt konflikt detection (409 Conflict)
- **Chain hash audit log** - nepopravljiv SHA-256 log (glej ADR-010)

2. Setup lokalnega okolja

Postopek za namestitev lokalnega razvojnega okolja. Celoten setup traja ~30 minut (od clone do delujočega dev serverja).

2.1 Predpogoji

Orodje	Verzija	Namig	Namestitev
Node.js	20.x LTS	Preveri: node --version	https://nodejs.org
Bun	1.1+	Preveri: bun --version	<code>curl -fsSL https://bun.sh/install bash</code>
Git	2.40+	Preveri: git --version	https://git-scm.com
VS Code	latest	Priporočeno IDE	https://code.visualstudio.com
OpenSSL	3.0+	Za FURS cert testing	<code>sudo apt install openssl</code>
Playwright	auto	Za E2E testi	<code>npx playwright install</code>

Tabela 2.1: Predpogoji za razvoj

2.2 Namestitev

```
# 1. Fork & kloniraj repozitorij
git clone https://github.com/TV0J-USERNAME/restaurantos.git
cd restaurantos

# 2. Dodaj upstream remote (za sync z glavnim repozitorijem)
git remote add upstream https://github.com/markec12345678/restaurantos.git

# 3. Namesti odvisnosti (Bun je hitrejši od npm)
bun install

# 4. Ustvari .env iz .env.example
cp .env.example .env

# 5. Nastavi environment spremenljivke v .env:
#   DATABASE_URL="postgresql://user:pass@host/db" # Neon ali lokalni PG
#   NEXTAUTH_SECRET="random-string-32-znakov"    # bun -e "console.log(crypto.randomUUID())"
#   FURS_ENV="test"
#   FURS_ALLOW_SIMULATION="true" # za razvoj brez cert-a

# 6. Generiraj Prisma client
bun run db:generate

# 7. Sinhroniziraj shemo z bazo
bun run db:push

# 8. (Opcija) Seed demo podatki
bun run dev
# Odpri http://localhost:3000/api/seed (kot admin)
```

```
# 9. Zaženi dev server
bun run dev

# 10. Odpri http://localhost:3000
#   Admin PIN: 1234
#   Super-admin PIN: 5555
```

2.3 VS Code priporočene razširitve

Razširitev	ID	Namig
ESLint	dbaeumer.vscode-eslint	Auto-fix na save
Prettier	esbenp.prettier-vscode	Format on save
Prisma	Prisma.prisma	Syntax highlight za .prisma
Tailwind CSS IntelliSense	bradlc.vscode-tailwindcss	Autocomplete za classe
TypeScript Vue Plugin	Vue.volar	Če delaš z .vue datotekami
Playwright Test	ms-playwright.playwright	E2E test runner
GitLens	eamodio.gitlens	Git blame in history
Error Lens	usernamehw.errorlens	Inline error prikaz

Tabela 2.2: VS Code razširitve

2.4 Debug setup

```
# VS Code launch.json (.vscode/launch.json):
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Next.js: dev",
      "type": "node",
      "request": "launch",
      "runtimeExecutable": "bun",
      "runtimeArgs": ["run", "dev"],
      "console": "integratedTerminal"
    },
    {
      "name": "Playwright E2E",
      "type": "node",
      "request": "launch",
      "runtimeExecutable": "bun",
      "runtimeArgs": ["run", "test:e2e"],
      "console": "integratedTerminal"
    }
  ]
}
```

3. Arhitektura in struktura projekta

RestaurantOS sledi modularni arhitekturi z jasno ločitvijo med frontend, backend in business logiko. Tukaj je pregled glavnih direktorijev.

3.1 Direktorijrska struktura

```
restaurantos/
├── src/
│   ├── app/ # Next.js App Router
│   │   ├── api/ # 229 API endpointov (serverless functions)
│   │   ├── (auth)/ # Avtentikacijske strani
│   │   ├── (dashboard)/ # Admin dashboard
│   │   ├── pos/ # POS aplikacija
│   │   ├── kds/ # Kitchen Display System
│   │   ├── waiter/ # Natakak mobilni vmesnik
│   │   ├── landing/ # Javna landing page
│   │   └── layout.tsx # Root layout
│   ├── components/ # 659 React komponent
│   │   ├── pos/ # POS moduli (orders, payments, inventory)
│   │   ├── ui/ # UI osnove (Button, Input, Modal)
│   │   ├── admin/ # Admin komponente
│   │   └── shared/ # Skupne komponente
│   ├── lib/ # Business logika
│   │   ├── auth-middleware/ # PIN auth, session, permissions
│   │   ├── furs/ # FURS API, ZOI, EOR, certifikati
│   │   ├── offline-orders/ # IndexedDB queue za naročila
│   │   ├── offline-furs/ # FURS offline queue
│   │   ├── accounting/ # Journal entries, Trial Balance
│   │   ├── websocket-client/ # WebSocket z auto-reconnect
│   │   ├── payment-gateways/ # Stripe, SumUp integracije
│   │   ├── outbox/ # Outbox pattern za async procesiranje
│   │   ├── audit/ # Chain hash audit log
│   │   ├── db.ts # Prisma client singleton
│   │   ├── logger.ts # Structured logging
│   │   └── validations/ # Zod sheme za input validacijo
│   ├── hooks/ # Custom React hooks
│   ├── types/ # TypeScript tipi
│   └── styles/ # Globalni CSS
├── prisma/
│   ├── schema.prisma # 92 Prisma modelov
│   ├── migrations/ # DB migracije
│   └── seed.ts # Demo podatki
├── public/ # Statične datoteke
│   ├── sw.js # Service Worker (v9)
│   ├── manifest.json # PWA manifest
│   └── icons/ # App ikone
├── tests/
│   ├── e2e/ # Playwright E2E testi
│   ├── unit/ # Vitest unit testi
│   └── setup.ts # Test setup
├── docs/ # Dokumentacija
├── scripts/ # Build/utility skripte
├── .env.example # Environment template
├── next.config.ts # Next.js konfiguracija
├── tailwind.config.ts # Tailwind konfiguracija
├── prisma.config.ts # Prisma konfiguracija
└── package.json # Odvisnosti in skripte
```

3.2 Ključni moduli

Modul	Lokacija	Odgovornost	Ključne datoteke
Auth	src/lib/auth-middleware/	PIN auth, session, RBAC	requireAuth, session, permissions
FURS	src/lib/furs/	Davčno potrjevanje	pkcs12-loader, zoi, verify-invoice
Orders	src/app/api/orders/	Naročila CRUD	route.ts, [id]/route.ts
Payments	src/app/api/payments/	Plačila	route.ts, stripe-intent
Inventory	src/app/api/inventory/	Zaloge	route.ts, restock, adjust
Audit	src/lib/audit/	Chain hash log	logger, verify-chain
PWA	public/sw.js	Offline mode	sw.js, manifest.json
WebSocket	src/lib/websocket-client/	Real-time	client, auto-reconnect

Tabela 3.1: Ključni moduli in njihova odgovornost

4. Kodni standardi

RestaurantOS sledi striktnim kodnim standardom za ohranjanje kvalitete kode. Vsi PR-ji morajo prestati lint, typecheck in teste pred merge.

4.1 TypeScript pravila

- **Strict mode** - vedno omogočen (tsconfig.json: "strict": true)
- **Brez `any`** - uporabi `unknown` + type guard ali specifičen tip
- **Brez `// @ts-ignore`** - uporabi `// @ts-expect-error` z razlago
- **Imena** - camelCase za spremenljivke, PascalCase za komponente/tipe/interface
- **Enums** - uporabi `as const` namesto `enum` (bolj tree-shakable)
- **Async** - vedno `async/await`, ne `.then().catch()`
- **Error handling** - vedno `try/catch` z typed error-ji

// ✅ PRAVILNO:

```
type PaymentMethod = 'cash' | 'card' | 'gift_card' as const
```

```
async function createPayment(data: PaymentInput): Promise<Payment> {
  try {
    const payment = await db.payment.create({ data })
    return payment
  } catch (error) {
    if (error instanceof Prisma.PrismaClientKnownRequestError) {
      throw new PaymentError('DATABASE_ERROR', error.message)
    }
    throw error
  }
}
```

// ❌ NAROBE:

```
async function createPayment(data: any): Promise<any> {
  const payment = await db.payment.create({ data }) // brez error handling
  return payment
}
```

4.2 React komponente

- **Server Components** - default (brez "use client") za statično vsebino
- **Client Components** - samo za interaktivne (onClick, useState, useEffect)
- **Props** - vedno typed interface, ne inline tipi
- **Hooks** - custom hooks v src/hooks/, prefiks `use`
- **State** - useState za lokalno, Zustand za globalno
- **Effects** - useEffect samo ko nujno, raje Server Components

// ✅ PRAVILNO - Server Component (default):

```
async function OrderList() {
  const orders = await db.order.findMany()
  return (
    <ul>
      {orders.map(o => <li key={o.id}>{o.number}</li>)}
    </ul>
  )
}
```

```

}

// ❗ PRAVILNO - Client Component (samo ko nujno):
'use client'
import { useState } from 'react'

interface PaymentButtonProps {
  orderId: string
  amount: number
}

export function PaymentButton({ orderId, amount }: PaymentButtonProps) {
  const [loading, setLoading] = useState(false)

  const handlePay = async () => {
    setLoading(true)
    try {
      await fetch('/api/payments', { method: 'POST', body: JSON.stringify({ orderId, amount }) })
    } finally {
      setLoading(false)
    }
  }

  return <button onClick={handlePay} disabled={loading}>Plačaj</button>
}

```

4.3 API Routes

- **Vedno `requireAuth()`** - razen za public endpointe (login, QR meni)
- **Zod validacija** - za vse inpute (body, query, params)
- **Rate limiting** - za občutljive endpointe (auth, FURS)
- **Error handling** - `handleApiError()` helper za konsistentnost
- **Idempotency** - za kritične operacije (plačila)

```

// ❗ PRAVILNO - API route pattern:
import { NextResponse } from 'next/server'
import { requireAuth } from '@lib/auth-middleware'
import { validateRequest, handleApiError } from '@lib/api-utils'
import { z } from 'zod'

const inputSchema = z.object({
  tableId: z.string().cuid(),
  items: z.array(z.object({
    menuItemId: z.string().cuid(),
    quantity: z.number().int().positive(),
  })).min(1),
})

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  try {
    // 1. Auth check
    const auth = await requireAuth(req, { permission: 'take_orders' })
    if (auth.error) return auth.error

    // 2. Input validation
    const { data, error } = await validateRequest(req, inputSchema, { maxBodySize: 32 * 1024 })
    if (error) return error

    // 3. Business logic
    const order = await createOrder(data, auth.session)
  } catch {
    // ...
  }
}

```

```
// 4. Response
return NextResponse.json(order, { status: 201 })
} catch (err) {
  return handleApiError(err, 'POST /api/orders', 'Napaka pri ustvarjanju naročila')
}
}
```

5. Testiranje

RestaurantOS uporablja tri nivoje testiranja: unit, integration in E2E. Vsi PR-ji morajo imeti teste za novo funkcionalnost.

5.1 Test hierarhija

Nivo	Orodje	Hitrost	Pokritost	Kdaj uporabiti
Unit	Vitest	Hitro (<1s/test)	Posamezne funkcije	Business logika, helpers, utils
Integration	Vitest + PGlite	Srednje (1-5s/test)	Modul interakcije	API routes z DB
E2E	Playwright	Počasno (10-30s/test)	Uporabniške poteze	Kritične poteze (log in, order, pay)

Tabela 5.1: Test hierarhija

5.2 Unit testi (Vitest)

```
# Poganjanje:
bun run test           # vsi unit testi
bun run test:watch     # watch mode
bun run test:coverage  # s pokritostjo

# Primer unit testa:
// tests/unit/zoi.test.ts
import { describe, it, expect } from 'vitest'
import { generateZOI } from '@lib/furs/crypto/zoi'

describe('generateZOI', () => {
  it('generira 32-znakovni MD5 hash', () => {
    const zoi = generateZOI({
      taxNumber: '12345678',
      issueDateTime: '2025-09-04T10:00:00Z',
      invoiceNumber: '001',
      businessPremiseId: 'PE1',
      electronicDeviceId: 'BLAG1',
      invoiceAmount: 49.90,
    })

    expect(zoi).toHaveLength(32)
    expect(zoi).toMatch(/^[a-f0-9]{32}$/)
  })

  it('vrne isti hash za isti input (deterministic)', () => {
    const input = { taxNumber: '12345678', /* ... */ }
    expect(generateZOI(input)).toBe(generateZOI(input))
  })
})
```

5.3 E2E testi (Playwright)

```
# Poganjanje:
bun run test:e2e           # vsi E2E testi
bunx playwright test --project=chromium # samo Chrome
bunx playwright test --headed          # z browser UI
bunx playwright test --debug           # z debugger

# Primer E2E testa:
// tests/e2e/critical-path.spec.ts
import { test, expect } from '@playwright/test'

test.describe('Ključna poteza: Order → Payment → FURS', () => {
  test('ustvari naročilo in plača z gotovino', async ({ page }) => {
    // 1. Prijava
    await page.goto('/')
    await page.fill('[data-testid=employee-id]', 'test-admin')
    await page.fill('[data-testid=pin]', '1111')
    await page.click('[data-testid=login]')

    // 2. Izberi mizo
    await page.click('[data-testid=table-5]')

    // 3. Dodaj artikel
    await page.click('[data-testid=menu-item-pizza]')
    await page.click('[data-testid=add-to-order]')

    // 4. Plačaj
    await page.click('[data-testid=pay]')
    await page.click('[data-testid=payment-cash]')
    await page.fill('[data-testid=received-amount]', '50')
    await page.click('[data-testid=confirm-payment]')

    // 5. Verificiraj
    await expect(page.locator('[data-testid=receipt]')).toBeVisible()
    await expect(page.locator('[data-testid=furs-eor]')).toBeVisible()
  })
})
```

5.4 Test pokritost cilji

Modul	Trenutna pokritost	Cilj	Prioriteta
FURS	85%	95%	Visoka (kritično)
Payments	78%	90%	Visoka (finančno)
Orders	82%	90%	Visoka (osnovno)
Auth	92%	95%	Visoka (varnost)
Inventory	65%	80%	Srednja
Reports	55%	75%	Srednja
Audit log	88%	95%	Visoka (compliance)

Tabela 5.2: Test pokritost po modulih

6. Contribution workflow

Postopek za prispevanje kode v RestaurantOS. Vsi prispevki gredo preko Pull Request-ov z minimum 1 reviewer.

6.1 Branching strategija

```
# Branch naming:
feat/<kratki-opis>      # nova funkcionalnost
fix/<kratki-opis>       # bug fix
chore/<kratki-opis>    # maintenance, deps
docs/<kratki-opis>     # dokumentacija
refactor/<kratki-opis>  # refaktoriranje
test/<kratki-opis>     # testi
security/<kratki-opis>  # varnostni fix
```

```
# Primeri:
feat/stripe-payment-intent
fix/furs-offline-queue-sync
chore/update-nextjs-16-1
docs/api-documentation
```

6.2 Commit convention

```
# Format:
type: kratki opis (imperative mood)

# Types:
feat:    nova funkcionalnost
fix:     bug fix
chore:   maintenance (deps, build, ci)
docs:   dokumentacija
test:   testi
refactor: refaktoriranje brez sprememb funkcionalnosti
perf:   performance izboljšava
security: varnostni fix

# Primeri:
feat: dodan Stripe PaymentIntent endpoint
fix: FURS offline queue sinhronizacija
chore: update Next.js na 16.1
docs: API dokumentacija za payments
test: E2E test za order flow
refactor: extract payment validation v helper
perf: index na orders.locationId
security: rate limit na auth endpoint
```

6.3 Pull Request postopek

```
# 1. Ustvari feature branch
git checkout -b feat/my-feature

# 2. Delaj in commit-aj
git add .
git commit -m "feat: dodana nova funkcionalnost"

# 3. Push na fork
git push origin feat/my-feature

# 4. Odpri Pull Request na GitHub
```

```
# - Naslov: "feat: kratki opis"
# - Opis: kaj, zakaj, kako testirati
# - Screenshot-i (če UI spremembe)
# - Link na issue (če obstaja)

# 5. Čakaj na code review (1-2 dni)
# - Addressiraj komentarje
# - Push fix-ev na isto branch (auto-updates PR)

# 6. Po odobritvi - SQUASH & MERGE
# - GitHub UI: "Squash and merge"
# - Edit commit message za finalno verzijo

# 7. Izbriši feature branch
git branch -d feat/my-feature
git push origin --delete feat/my-feature
```

6.4 PR checklist

- ☐ Koda opravi `bun run lint` (brez errorjev)
- ☐ Koda opravi `bun run typecheck` (brez errorjev)
- ☐ Vsi testi opravijo `bun run test`
- ☐ E2E testi opravijo `bun run test:e2e` (za kritične spremembe)
- ☐ Ni `console.log` v produkciji (uporabi `logger`)
- ☐ Ni `any` tipov (uporabi pravilne tipe)
- ☐ Input validacija z Zod na novih endpointih
- ☐ `requireAuth()` na novih API endpointih
- ☐ Rate limiting na občutljivih endpointih
- ☐ Testi za novo funkcionalnost (unit + integration)
- ☐ Dokumentacija posodobljena (JSDoc, README)
- ☐ Ni hardcoded secretov ali API ključev
- ☐ Spremembe so backward compatible (ali pa major version bump)

7. Debug in troubleshooting

7.1 Pogoste težave in rešitve

Težava	Vzrok	Rešitev
Prisma client error	Sprememba sheme brez generate	bun run db:generate
Module not found	Netočni import path	Preveri @/ alias v tsconfig.json
Hydration mismatch	Server/Client razlika	Dodaj "use client" ali useEffect
Type error na build	TypeScript strict	Popravi tip ali dodaj @ts-expect-error
FURS timeout	Cert geslo narobe	Preveri FURS_CERT_PASSWORD
Stripe webhook fail	Webhook secret narobe	Preveri STRIPE_WEBHOOK_SECRET
DB connection exhausted	Preveč concurrent conn.	Restart Neon ali povečaj pool
Build fail na Vercel	Memory limit	Povečaj function memory na 2048MB
E2E test flaky	Race condition	Dodaj waitForSelector

Tabela 7.1: Pogoste težave in rešitve

7.2 Debug orodja

```
# 1. Prisma Studio (DB GUI):
bunx prisma studio
# Odpri http://localhost:5555

# 2. React DevTools:
# Namesti Chrome razširitev "React Developer Tools"

# 3. Network tab (Chrome DevTools):
# - Preveri API klice (status, payload, timing)
# - Filter: /api/ za samo API klice

# 4. Sentry (lokalno):
# - Postavi SENTRY_DSN v .env
# - Sproži napako in preveri Sentry dashboard

# 5. Logging:
# - Uporabi `logger.info()` namesto `console.log()`
# - Logi so vidni v terminalu (dev) in Vercel dashboard (prod)

# 6. Debug mode:
DEBUG=* bun run dev # verbose logi

# 7. Playwright debug:
bunx playwright test --debug
```

- # - Odpri Playwright Inspector
- # - Step-through debugging

7.3 Performance profiling

- # 1. Next.js bundle analyzer:

- bun run build

- # Preveri .next/analyze/ za bundle breakdown

- # 2. React Profiler:

- # - Namesti React DevTools Profiler

- # - Record session in analiziraj re-renders

- # 3. Lighthouse:

- # - Chrome DevTools → Lighthouse

- # - Run audit za Performance, Accessibility, Best Practices

- # 4. Vercel Analytics:

- # - Dashboard → Analytics

- # - Preveri Web Vitals (LCP, FID, CLS)

- # 5. Database slow queries:

- # - Neon dashboard → Metrics

- # - Preveri pg_stat_statements za top queries

8. Deployment

RestaurantOS se deploy-a na Vercel avtomatsko ob push-u na main branch. Staging environment se deploy-a ob push-u na druge branch-e.

8.1 Environment-i

Environment	URL	Branch	DB	Naime
Produkcija	restaurantos.app	main	Neon prod	Produkcijška uporaba
Staging	restaurantos-staging.vercel.app	any branch	Neon staging	Testiranje pred prod
Preview	.vercel.app	PR branch	Neon staging	Code review test
Lokalno	localhost:3000	working dir	PGLite/Neon dev	Razvoj

Tabela 8.1: Environment-i

8.2 Deploy postopek

```
# Avtomatski deploy (main → production):
git push origin main
# Vercel avtomatsko zazna push in zaženi build
# Deploy traja 2-5 minut
# Preview na https://restaurantos-<commit>.vercel.app
# Produkcija na https://restaurantos.app (po promote)

# Ročni deploy (Vercel CLI):
vercel                # deploy na preview
vercel --prod         # deploy na produkcijo

# Promote preview v produkcijo:
# Vercel Dashboard → Deployments → izberi → "Promote to Production"

# Rollback:
# Vercel Dashboard → Deployments → prejšnji → "Promote to Production"
```

8.3 Environment variables

Spremenljivka	Produkcija	Staging	Lokalno
DATABASE_URL	Neon prod URL	Neon staging URL	Neon dev ali PGLite

Spremenljivka	Produkcija	Staging	Lokal no
NEXTAUTH_SECRET	strong-random	strong-random	random
FURS_ENV	production	test	test
FURS_CERT_PATH	./certs/furs-prod.p12	./certs/furs-test.p12	-
FURS_ALLOW_SIMULATION	false	false	true
STRIPE_SECRET_KEY	sk_live_...	sk_test_...	sk_test_...
STRIPE_WEBHOOK_SECRET	whsec_live_...	whsec_test_...	-
SENTRY_DSN	production DSN	staging DSN	-
NEXT_PUBLIC_VAPID_PUBLIC_KEY	live key	test key	-

Tabela 8.2: Environment variables po environment-u

8.4 Pred-deploy checklist

- ☐ Vsi testi opravijo (unit + E2E)
- ☐ TypeScript build brez napak
- ☐ ESLint brez napak
- ☐ Code review odobren
- ☐ Migration aplicirana na staging in testirana
- ☐ DB backup narejen (Neon snapshot)
- ☐ Sentry release tag nastavljen
- ☐ Slack #deploys notified
- ☐ Deploy v off-peak uri (22:00-06:00)
- ☐ On-call oseba obveščena

9. Viri in nadaljnje učenje

9.1 Notranja dokumentacija

Dokument	Lokacija	Namen
Architecture	docs/ARCHITECTURE.md	Sistemiški diagram, moduli
Code Review Report	docs/CODE-REVIEW-REPORT.md	85 checks, 11 fixes
Production Launch	docs/PRODUCTION-LAUNCH-CHECKLIST.md	Launch checklist
Client Onboarding	docs/CLIENT-ONBOARDING-GUIDE.md	Stranka navodila
ADR Zbirka	download/RestaurantOS-ADR-Zbirka.pdf	12 arhitekturnih odločitev
API Dokumentacija	download/RestaurantOS-API-Dokumentacija.pdf	60+ endpointov
OpenAPI spec	download/openapi.yaml	SDK generacija
Production Runbook	download/RestaurantOS-Production-Runbook.pdf	Operacije
P0 Sprint Plan	download/RestaurantOS-P0-Sprint-Plan.xlsx	42 nalog

Tabela 9.1: Notranja dokumentacija

9.2 Zunanji viri

- **Next.js 16 docs:** <https://nextjs.org/docs>
- **React 19 docs:** <https://react.dev>
- **Prisma docs:** <https://www.prisma.io/docs>
- **Neon docs:** <https://neon.tech/docs>
- **Tailwind CSS 4:** <https://tailwindcss.com/docs>
- **Vercel docs:** <https://vercel.com/docs>
- **Stripe docs:** <https://stripe.com/docs>
- **FURS docs:** <https://edavki.durs.si/Documentation>
- **Playwright docs:** <https://playwright.dev>
- **Vitest docs:** <https://vitest.dev>

9.3 Kontakti

Vloga	Kontakt	Kdaj kontaktirati
Tech Lead	robert@restaurantos.app	Arhitekturne odločitve, blockers
Backend Dev	marko@restaurantos.app	API, baza, FURS, Stripe
Frontend Dev	ana@restaurantos.app	UI, PWA, komponente
QA	peter@restaurantos.app	Testiranje, E2E, regression
Slack #dev	restaurantos.slack.com/#dev	Hitra vprašanja, help
Slack #incidents	restaurantos.slack.com/#incidents	Produksijski incidenti
GitHub Issues	github.com/markec12345678/restaurantos/issues	Bug reporti, feature requests

Tabela 9.2: Kontakti za razvoj

DOBRODOŠLI V EKIP!

Ta vodič je živ dokument - če najdeš napako ali manjkajočo sekcijo, odpri PR z popravkom. Tvoje izkušnje kot nov developer so neprecenljive za izboljšanje tega vodiča za prihodnje člane ekipe. Srečno in dobrodošli v RestaurantOS ekipi!